

Sistema de Gestão Comercial - Entrega 1

Projeto acadêmico de Desenvolvimento Web

Contexto escolhido: Papelaria Alfa

Objetivo	Modelar um sistema web para controlar clientes, produtos, vendas, usuários e relatórios em um pequeno negócio.
Tecnologia proposta	Django + Django REST Framework + SQLite + JWT + Bootstrap.
Perfis de acesso	ADMIN e FUNCIONARIO.

1. Descrição do sistema

O sistema proposto atende uma papelaria de pequeno porte que hoje controla clientes, produtos e vendas de forma manual ou em planilhas. O objetivo é centralizar o cadastro de clientes, o controle de estoque, o registro de vendas e a emissão de relatórios em uma única aplicação web. O sistema também deve oferecer autenticação por token, API REST e interface web consumindo essa API, conforme exigido na disciplina.

2. Objetivos do sistema

Os objetivos principais são reduzir erros de digitação, evitar vendas com estoque insuficiente, manter histórico confiável de vendas, facilitar o atendimento ao cliente e gerar relatórios para tomada de decisão. O sistema foi desenhado para ser simples, escalável e compatível com a evolução das próximas entregas.

3. Requisitos funcionais

- RF01 - Realizar login de usuário e gerar token JWT.
- RF02 - Permitir cadastro, edição, listagem e remoção de clientes.
- RF03 - Validar CPF único por cliente.
- RF04 - Validar formato de e-mail do cliente.
- RF05 - Impedir remoção de cliente com vendas registradas.
- RF06 - Permitir cadastro, edição, listagem e remoção de produtos.
- RF07 - Impedir preço negativo e controlar estoque mínimo.
- RF08 - Impedir registro de venda com estoque insuficiente.
- RF09 - Registrar venda com cliente, usuário responsável e itens.
- RF10 - Calcular valor total da venda automaticamente.
- RF11 - Atualizar estoque após a confirmação da venda.
- RF12 - Gerar relatório de vendas por período.
- RF13 - Gerar relatório de vendas por cliente.
- RF14 - Exibir representação gráfica das vendas anuais.
- RF15 - Controlar acesso por perfil ADMIN e FUNCIONARIO.

4. Requisitos não funcionais

- RNF01 - Arquitetura em camadas para facilitar manutenção e testes.

- RNF02 - API REST retornando JSON em todos os endpoints.
- RNF03 - Interface web desenvolvida em Django, com navegação simples.
- RNF04 - Persistência em banco de dados relacional.
- RNF05 - Senhas armazenadas de forma criptografada.
- RNF06 - Token com expiração configurável.
- RNF07 - Código versionado no GitHub com histórico organizado.
- RNF08 - Estrutura preparada para crescimento sem reescrever o núcleo do sistema.
- RNF09 - Respostas com tratamento de erro consistente e mensagens claras.
- RNF10 - Tempo de resposta adequado para operações comuns de cadastro e consulta.

5. Arquitetura proposta

A solução adota arquitetura em camadas, separando responsabilidades em apresentação, aplicação, domínio e infraestrutura. A interface web em Django consome a API REST da própria aplicação. A camada de aplicação concentra regras de negócio, a camada de domínio representa as entidades principais e a camada de infraestrutura realiza persistência, autenticação e integração com o banco de dados.

Camada	Responsabilidade	Exemplos
Apresentação	Telas web, formulários, navegação e consumo da API	Templates Django, views e Bootstrap
API	Endpoints REST, serialização e validação inicial	AuthView, ClienteViewSet, ProdutoViewSet e VendaViewSet
Aplicação/Serviços	Regras de negócio e orquestração dos casos de uso	VendaService, ClienteService e RelatorioService
Domínio	Entidades centrais e regras conceituais	Cliente, Produto, Venda, ItemVenda e Usuario
Infraestrutura	ORM, banco, autenticação JWT, logs e configurações	Django ORM, SQLite ou PostgreSQL e SimpleJWT

6. Padrões de projeto escolhidos

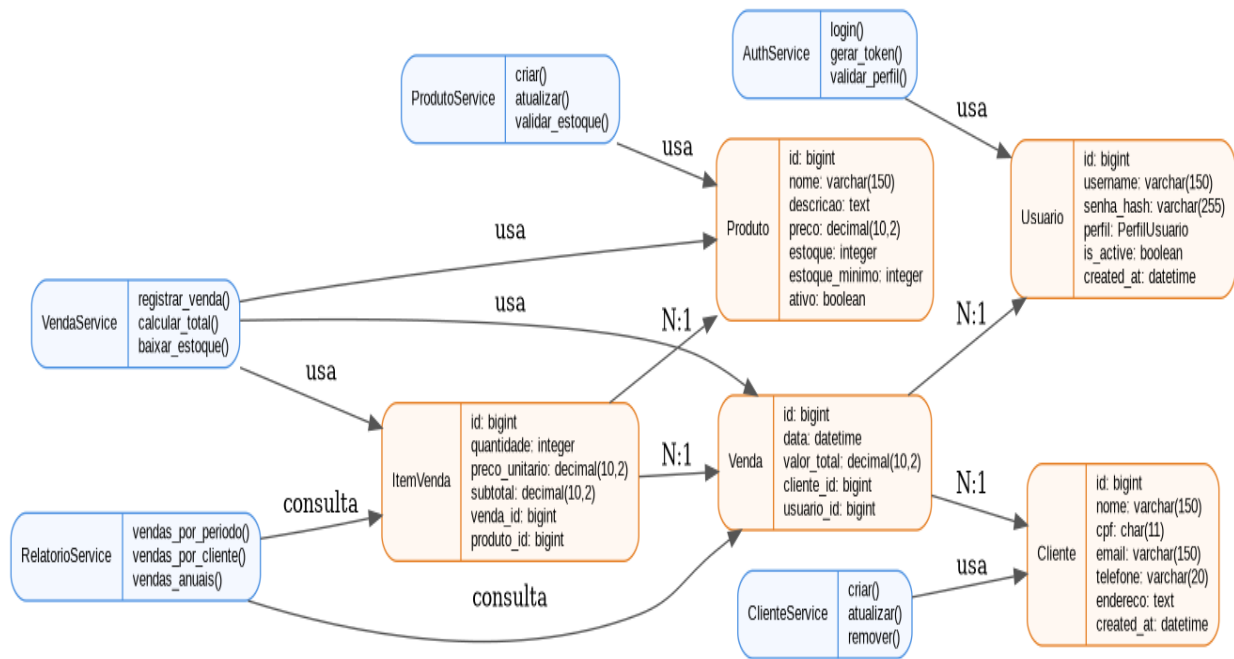
- **MVC:** adequado ao Django para organizar models, views e templates.
- **Service Layer:** centraliza regras de negócio mais sensíveis, como venda, cálculo total e baixa de estoque.
- **Repository via ORM:** o Django ORM abstrai o acesso a dados, reduz repetição e melhora legibilidade.
- **DTO/Serializer:** serializers da API controlam entrada e saída JSON com validação consistente.
- **RBAC simples:** controle de acesso baseado em papéis ADMIN e FUNCIONARIO.

7. Justificativa das decisões técnicas

Django foi escolhido por já oferecer estrutura madura para autenticação, templates, ORM e segurança básica. O Django REST Framework facilita a construção da API REST e o SimpleJWT atende a exigência de autenticação por token. SQLite é suficiente para a fase inicial e simplifica a configuração local; caso a equipe precise, o projeto pode migrar depois para PostgreSQL sem alterar o desenho principal. A separação por camadas ajuda a alcançar os critérios de manutenibilidade, organização e clareza pedidos na avaliação.

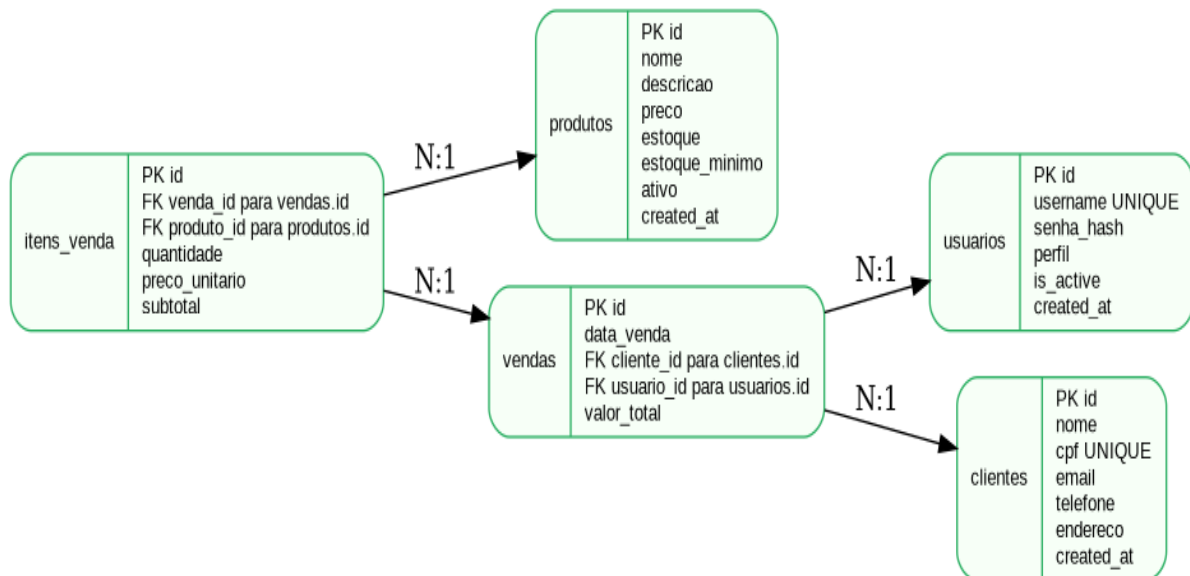
8. Diagrama de classes

O diagrama abaixo representa as principais entidades do domínio e os serviços centrais da aplicação.



9. Diagrama lógico do banco de dados

O banco contém as tabelas mínimas exigidas: usuarios, clientes, produtos, vendas e itens_venda.



10. Script de criação do banco de dados

Trecho inicial do script SQL. O arquivo completo está salvo em database/schema.sql.

```
-- Sistema de Gestão Comercial para Pequenos Negócios
-- Contexto escolhido: Papeleria Alfa
CREATE TABLE usuarios (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username VARCHAR(150) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL,
  perfil VARCHAR(20) NOT NULL CHECK (perfil IN ('ADMIN', 'FUNCIONARIO')),
  is_active BOOLEAN NOT NULL DEFAULT 1,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE clientes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nome VARCHAR(150) NOT NULL,
  cpf CHAR(11) NOT NULL UNIQUE,
  email VARCHAR(150) NOT NULL,
```

```

    telefone VARCHAR(20) NOT NULL,
    endereco TEXT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE produtos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome VARCHAR(150) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10,2) NOT NULL CHECK (preco >= 0),
    estoque INTEGER NOT NULL DEFAULT 0 CHECK (estoque >= 0),

```

11. Estrutura inicial do repositório GitHub

Caminho	Finalidade
README.md	Resumo do projeto, stack, entregas e instruções iniciais.
docs/	PDF da entrega 1, diagramas e documentação complementar.
database/schema.sql	Script de criação do banco de dados.
src/	Aplicação Django que será implementada na entrega 2.
.gitignore	Arquivos e pastas que não devem subir para o GitHub.

12. Casos de uso considerados nesta fase

- Autenticar usuário.
- Gerenciar clientes.
- Gerenciar produtos.
- Registrar venda.
- Consultar vendas por período.
- Consultar vendas por cliente.
- Visualizar gráfico anual de vendas.

13. Conclusão

A entrega 1 estabelece uma base consistente para as próximas etapas, cobrindo contexto do problema, requisitos, arquitetura, modelagem de classes, modelagem do banco e estrutura inicial de versionamento. A próxima fase deve focar na implementação do backend, da API REST, das regras de venda e do consumo da API pela interface web.